

컴퓨터비전 응용 2주차 요약본

6. 신경망 스타일 전이(Neural Style Transfer)

6.1 Neural Style Transfer란?

- 한 이미지의 내용(Content)과 다른 이미지의 스타일(Style)을 결합해 새로운 이미지를 생성하는 기법
- Content image + Style image → 합성된 Generated image 생성
 - **Content image**: 사진처럼 객체 배치, 구조가 중요한 이미지
 - **Style image**: 그림처럼 색감, 붓터치, 질감이 중요한 이미지
 - 구조는 content처럼, 표현은 style처럼
 - 생성할 이미지를 하나의 학습 파라미터로 취급하여, Content Loss와 Style Loss를 최소화하도록 이미지 자체를 업데이트

6.2 Model Architecture

- NST는 사전 학습된 CNN(주로 VGG16/VGG19)을 기반으로 사용
- 고정된 Feature Extractor로 VGG16 사용 (가중치 업데이트 X)
- 중간 레이어에서 Content 및 Style Feature 추출
 - 낮은 레이어: Edge, corner
 - 중간 레이어: Texture, style pattern
 - 높은 레이어: 객체 구조, semantic 정보
- 새로운 이미지(generated image)를 학습하며, 두 특성을 동시에 맞춤
- 기존 CNN 구조를 입력 특징 추출용으로만 사용하고 생성 이미지가 학습대상으로서 최적화되는 방식

6.3 Content Representation

- Content는 이미지의 구조적 특징(형태, 객체 배열)을 의미
 - CNN 특정 레이어의 feature map을 그대로 사용하여 비교
 - 일반적으로 VGG16의 conv4_2처럼 구조적 특징이 잘 추출되는 중간 레이어 사용

- Content Loss 공식

$$L_{content} = \|F_{content} - F_{generated}\|_2$$

6.4 Style Representation (Gram Matrix)

- Style은 이미지의 색감, 질감, 패턴, 즉 채널 간 상관관계를 모델링
- Gram Matrix란?
 - Feature map 채널들 간의 **내적(dot product)** 관계를 담은 행렬
 - 특정 스타일의 패턴 반복, 질감을 정량화하는 방식
 - spatial 정보를 버리고 channel 간 상관관계만 남김 → 스타일 정보만 추출 가능
- Gram Matrix 공식

$$G = FFT$$

- Style Loss

$$L_{style} = l \sum w_l \|G_{lstyle} - G_{lgenerated}\|_2$$

6.5 NST의 전체 Loss 구조

- 최종 Loss는 다음 두 항의 조합

$$L_{total} = \alpha L_{content} + \beta L_{style}$$

- α (alpha): Content 비중
- β (beta): Style 비중
 - β 가 크면 결과물이 더 화풍에 가까워짐

6.6 NST 학습 과정

1. 초기 Generated Image 설정

- Content image 그대로 사용하거나
- Random noise로 시작 가능 (빠르게 수렴하지는 않음)

2. Forward Pass

- Content, Style, Generated image를 CNN에 통과시키고
- 각 레이어에서 feature map과 Gram Matrix를 추출

3. Loss 계산

- content loss
- style loss

4. Backpropagation

- CNN 가중치가 아니라 생성된 이미지 tensor 자체에 대한 gradient를 계산

5. Optimizer로 이미지 업데이트

많이 사용되는 최적화 기법:

- L-BFGS (전통적 NST에서 가장 많이 사용)
 - Adam

6.7 Neural Style Transfer 코드

- 스타일 이미지와 콘텐츠 이미지를 PyTorch 텐서로 변환

```
# 이미지 파일 경로를 통해, (1, 3, H, W) 형태의 텐서(tensor)로 변환하는 함수
def image_to_tensor(image_filepath, image_dimension=128):
    # image_dimension: 크기 제한
    # PIL을 사용하여 이미지 로드, 흑백/투명 채널을 제거하고 항상 3 채널(RGB)로 변환
    img = Image.open(image_filepath).convert('RGB')

    # resize 기준 설정: 최대 너비/높이를 기준으로 Resize할 크기를 설정
    if max(img.size) <= image_dimension:
        img_size = max(img.size)
    else:
        img_size = image_dimension

    torch_transformation = torchvision.transforms.Compose([
        torchvision.transforms.Resize(img_size), # 가장 큰 축을 img_size 리사이즈
        torchvision.transforms.ToTensor() # (H, W, C) → (C, H, W) 로 변환
    ])
```

```
# .unsqueeze(0): 배치 차원 추가/ (3, H, W) → (1, 3, H, W)
img = torch_transformation(img).unsqueeze(0)
```

```
# device: GPU or CPU로 이동, dtype을 float으로 설정
return img.to(device, torch.float)
```

• Gram Matrix계산

- 스타일 이미지는 원본 이미지와 달리 색감·질감 등 **특정 레이어의 채널 간 상관관계**로 표현되며, 이를 정량화한 것이 Gram Matrix라고 볼 수 있음

```
# 입력된 CNN feature map 텐서에서 Gram Matrix를 계산
# 이 행렬은 NST에서 style loss 계산 시 사용
# 입력 텐서 ip는 shape = (N, C, H, W)
# N: 배치 크기, C: 채널 수, H, W: 세로, 가로 크기(feature map의 공간 크기)

def gram_matrix(ip):
    num_batch, num_channels, height, width = ip.size()
    # 텐서를 2D 행렬로 펼침(flatten)
    # ->각 행은 하나의 채널에 해당하는 feature map을 벡터화, shape = (N × C, H × W)
    feats = ip.view(num_batch * num_channels, width * height)

    # feats × featsT (내적) → 채널 간 상관관계 행렬 계산, shape = (N × C, N × C)
    gram_mat = torch.mm(feats, feats.t()) #각 원소 Gij는 채널 i와 채널 j 간의 유사도
    # Gram Matrix는 입력 크기에 따라 스케일이 달라지므로 정규화,
    # 나누는 값은 총 feature 수 = N × C × H × W
    return gram_mat.div(num_batch * num_channels * width * height)

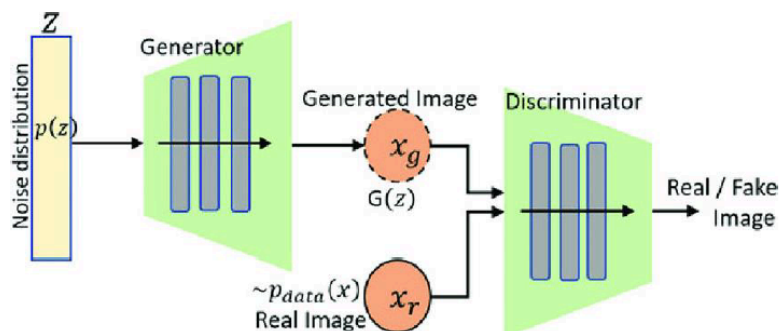
-> 레이어마다 feature 크기가 달라도 균형 있게 style loss에 기여
```

7. 이미지 생성형_GAN(Generative Adversarial Networks)

7.1 GAN (Generative Adversarial Networks)이란?

- 서로 다른 두 개의 네트워크를 적대적(adversarial) 학습, 실제 데이터와 비슷한 데이터를 생성하는 모델
- 생성자(Generator, G)와 판별자(Discriminator, D) 두 개의 모델이 경쟁 구조로 학습
- 핵심 개념
 - 생성자(G): 가짜 이미지를 생성 (노이즈 → 이미지)
 - 판별자(D): 입력 이미지가 진짜인지 가짜인지 판단
- 목표
 - G: D를 속일 만큼 진짜 같은 이미지 생성
 - D: G가 만든 이미지를 구분해냄
- 구조
 - 입력: **노이즈 벡터 z**
 - G: z 를 실제 같은 이미지로 변환
 - D: 이미지가 진짜인지 판별
- 구글브레인의 Ian Goodfellow에 의해 2014년도에 해당 개념 도입
- **Vanilla GAN (기본 GAN 구조)**
 - 완전 연결층 (Fully Connected Layers) 기반
 - MNIST 등 **저해상도 이미지**에 생성에 사용
 - 공간 구조(spatial info)를 보존하기 어려움 → 고해상도 생성 어려움

▶ Vanilla GAN architecture



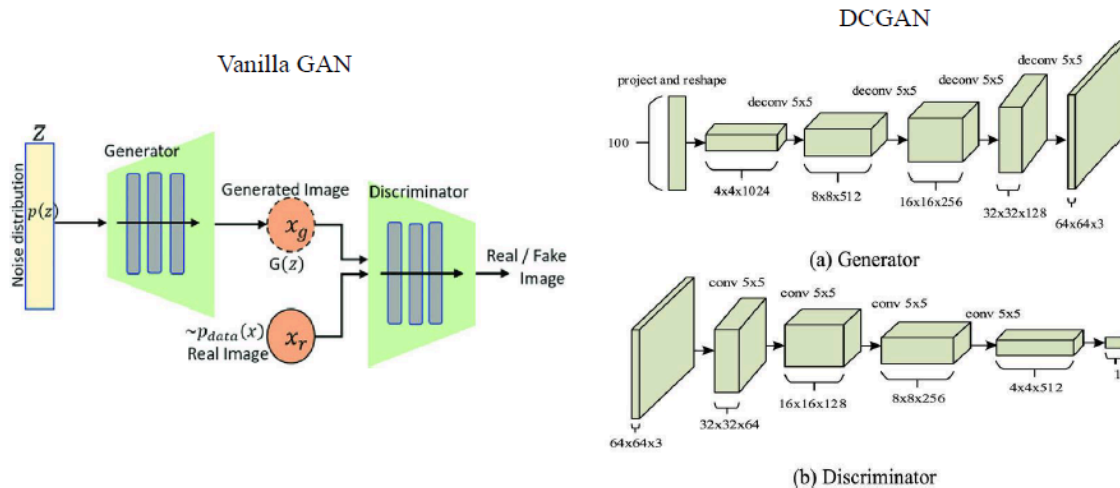
7.2 GAN의 주요 문제점

- **Mode Collapse**
 - G가 "특정한 형태의 이미지"만 반복해서 생성
 - 다양성이 부족해짐
- **학습 불안정**
 - G와 D가 동시에 학습되므로 균형 유지가 어려움
 - 하나가 너무 강해지면 학습이 망가지는 경우가 있음
- **평가 지표 부족**
 - 생성 품질을 정량적으로 평가하기 어려움
 - Inception Score, FID 등이 후에 등장

7.3 DCGAN (Deep Convolutional GAN)

- 합성곱 신경망 기반의 GAN 구조
- Vanilla GAN보다 더 깊은 신경망 사용 (CNN)
- CNN 기반으로 고해상도 이미지 생성 가능
- **DCGAN의 벡터 연산**
 - 잠재 벡터 z 의 조작이 의미 있는 변화로 이어짐
 - 예시1) 웃는 얼굴 ↔ 안 웃는 얼굴
 - 예시2) "남성 → 여성" 방향으로 벡터 차 연산 가능
 - Latent space의 의미론적 표현 가능

DCGAN architecture



<그림 설명>

[왼쪽: Vanilla GAN Architecture]

- 완전연결층 (fully connected layers) 중심
- 이미지 생성에 적합하지 않은 단순한 MLP 구조 사용
- 이미지의 공간적 구조(spatial structure)를 잘 보존하지 못함

[오른쪽: DCGAN Architecture]

- Vanilla GAN의 **fully connected layers** → **CNN (Convolutional Neural Network)** 구조로 대체
- 특히, 이미지 생성/분류에 특화된 합성곱 계층 사용
- **(a) Generator (DCGAN)**
 - 구조
 - **입력:** z (예: 100차원 노이즈 벡터)
 - **project and reshape:** 벡터를 4x4×1024 feature map으로 변형
 - 여러 단계의 **Transposed Convolution (Deconvolution)** 사용
 - **최종 출력:** 64x64×3 크기의 RGB 이미지
 - 특징
 - **deconv 5x5**: 커널 크기 5x5의 transposed convolution
 - 점점 해상도가 커지는 방향 (**4x4** → **8x8** → **16x16** → **32x32** → **64x64**)

- 마지막 층은 **tanh** 활성화 함수 사용 (픽셀 값 -1~1)
- **Discriminator (DCGAN)**
 - 구조
 - **입력**: 64x64x3 이미지
 - 여러 층의 **Convolution + LeakyReLU + BatchNorm**
 - 점점 해상도 줄이며 feature 추출
 - **최종 출력**: 1개의 스칼라 값 → 진짜/가짜 여부 확률
 - 특징
 - **conv 5x5**: 커널 크기 5x5의 convolution
 - 출력이 0에 가까우면 "가짜", 1에 가까우면 "진짜"
 - 마지막에는 **sigmoid** 함수 사용

7.4 Conditional GAN (cGAN)

- 조건부 생성이 가능한 GAN (예: 특정 숫자나 클래스 지정)
- 입력: **[z, label]** 형식
- 예시: MNIST에서 "숫자 7만 생성해줘" → 7이라는 label로 cGAN 생성
- **cGAN Loss Function**
 - label 정보를 포함한 GAN 구조
 - 조건 y 를 G와 D에 같이 입력

7.5 Vanilla GAN vs DCGAN 비교

항목	Vanilla GAN	DCGAN
네트워크 구조	MLP (fully connected)	CNN 기반
이미지 처리	공간 정보 손실 큼	공간 정보 잘 보존
Generator 업샘플링	없음	Deconvolution 사용
Discriminator 다운샘플링	없음	Convolution 사용
성능	낮음	고해상도 이미지 생성 가능

7.6 Pix2Pix (Image-to-Image Translation)

- 입력 이미지 → 원하는 형태의 출력 이미지로 변환을 학습
 - 구조
 - **Generator: U-Net 기반 Encoder-Decoder**
 - Encoder: 이미지를 feature로 압축
 - Decoder: 다시 이미지로 복원
 - Skip-connection: 로컬 디테일 보존(엣지 정보 유지)
 - **Discriminator: PatchGAN**
 - 전체 이미지가 아닌 **$N \times N$ patch** 단위로 판별
 - 이미지의 지역 정보(local texture) 검출에 강함
 - 고해상도 판별에 효율적
 - **Pix2Pix Loss**
 - **GAN Loss**
 - **L1 Reconstruction Loss** (정답 이미지와의 차이를 직접 줄이는 역할)
 - 두 개를 합쳐 최종 loss로 사용
- ➔ GAN의 “사실감” + L1의 “정확한 형태 재현” 동시 달성

8. 이미지 캡셔닝 (CNN_LSTM)

8.1 이미지 캡셔닝

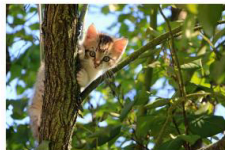
- 입력 이미지를 바탕으로 이미지의 내용을 설명해주는 문장을 만드는 것

Image Captioning: Example Results

Captions generated using [neuraltalk2](#)
All images are CC0 Public domain: [cat](#)
[suitcase](#) [cat tree](#) [dog bear surface](#)
[giraffe](#) [motorcycle](#)



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court

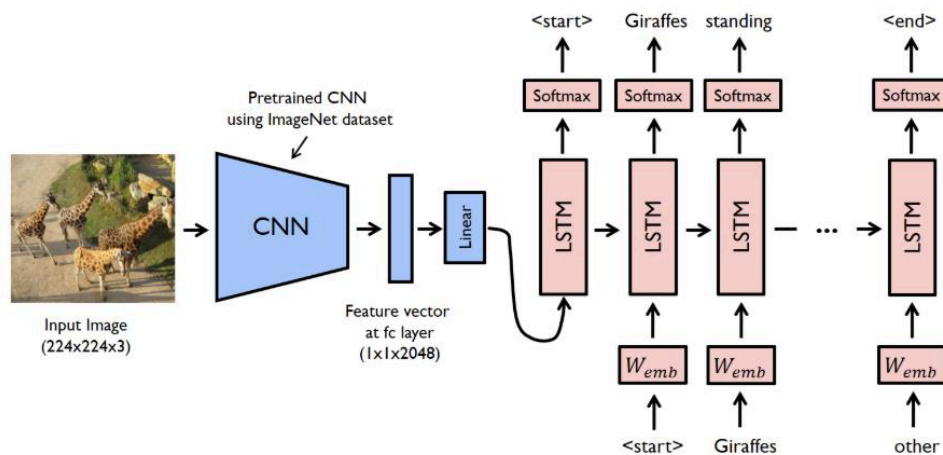


Two giraffes standing in a grassy field



A man riding a dirt bike on a dirt track

- 사전 훈련된 CNN으로 이미지의 내용을 이해하고, LSTM이 그 내용을 바탕으로 문장을 생성하는 인코더-디코더 구조



• 이미지 캡셔닝 단계

1. 이미지 입력

입력 이미지는 딥러닝 모델의 입력으로 사용. 이 이미지는 캡션에 사용될 semantic 정보를 추출하기 위한 feature map으로 변환됨

2. 이미지 특성 추출 (CNN 모델 사용)

이미지에서 semantic 정보를 추출하기 위해 딥러닝 모델을 사용. 딥러닝 모델을 이용하여 이미지의 다양한 특성을 학습하고 semantic 정보를 벡터로 추출

3. 문맥 학습 (순환 신경망(RNN, LSTM) 또는 Transformer 모델 사용)

추출된 semantic 정보를 벡터 기반으로 모델이 문맥을 학습. 이 단계에서는 이미지 특성을 시퀀스로 변환하여 문맥을 이해

4. 자연어 설명 생성 (순환 신경망 또는 Transformer 모델 사용)

학습된 모델이 이미지의 특성과 문맥을 바탕으로 이미지에 대한 캡션을 생성

8.2 Pytorch CNN과 LSTM 신경망 구현

- 이미지에서 특징을 추출하는 CNN 모델과 순차적 예측을 수행하는 LSTM 계층으로 구성
- CNN 모델의 마지막 은닉 계층이 LSTM의 입력으로 사용
- LSTM은 텍스트를 생성하는 디코더 역할

8.3 텍스트 인코딩

- 캡션(문장)을 모델이 이해할 수 있도록 숫자로 변환
 - 예시) hello world → 4, 7
- 일반적으로 토큰화(Tokenization) → 단어 사전 생성 → 단어를 인덱스로 매핑
 - 토큰화: "hello world" → ['hello', 'world']
 - 단어사전: {'hello':4, 'world':7}
 - 숫자 시퀀스로 변환: [4, 7]

8.4 데이터 전처리

- 이미지 데이터 전처리
 - CNN 입력 크기에 맞게 리사이즈
 - Normalize, ToTensor 등 변환 수행
- 캡션 데이터 전처리
 - 각 단어를 숫자로 변환
 - <start>, <end> 토큰 삽입
 - 길이 맞춰 패딩 처리

8.5 CNN-LSTM 모델 학습

- 입력: (이미지, 캡션)
- CNN으로 이미지 feature vector 추출
- LSTM에 캡션의 앞부분(teacher forcing 방식)을 넣어 다음 단어를 예측하게 학습

- 손실함수: **CrossEntropyLoss**
- 옵티마이저: Adam 등 사용
- **학습Flow**
 1. 이미지 CNN forward → feature vector
 2. 캡션의 실제 단어를 입력으로 LSTM forward
 3. 모델이 예측한 단어와 실제 단어 비교 → Loss 계산
 4. Backpropagation으로 파라미터 업데이트

8.6 훈련된 모델로부터 캡션 생성하기

학습 후 새로운 이미지 입력 시:

1. CNN으로 특징 추출
2. LSTM에 <start> 입력
3. 한 단어씩 생성
4. <end> 토큰이 나올 때까지 반복

단어 생성에는 **Greedy Search** 또는 **Beam Search** 사용 가능

- Greedy: 가장 높은 확률 단어 단순 선택
- Beam Search: 여러 후보 문장을 동시에 고려하여 더 자연스러운 문장 생성

9. 3D 렌더링(Rendering)

9.1 Neural Radiance Fields (NeRF)

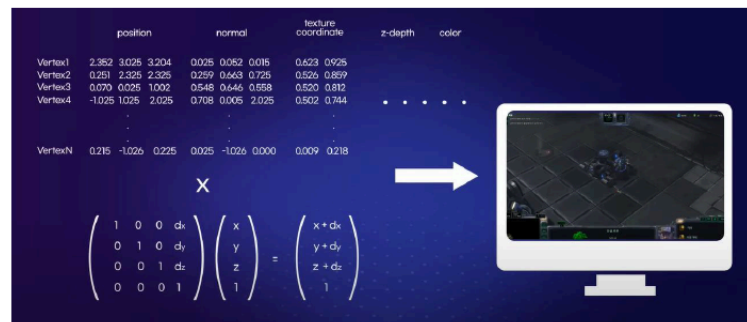
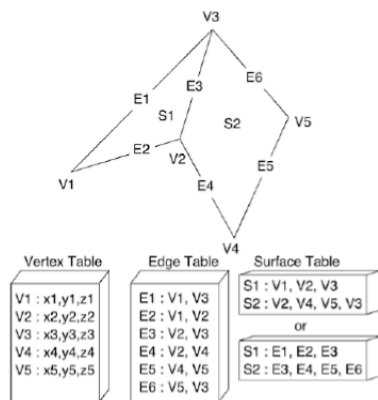
- NeRF는 3D 공간의 연속적인 장면(continuous scene representation) 을 학습하는 모델
- 입력 : 5D 벡터(공간좌표 x, y, z + 시선 방향 θ, ϕ)
- 출력 : RGB 색상 + 밀도(σ : opacity / density)
- NeRF는 실제 여러 각도에서 찍힌 이미지들을 기반으로 볼륨 렌더링(Volume Rendering) 원리를 이용해 장면을 재구성
- 핵심 아이디어

1. 3D 장면을 voxel이나 mesh로 저장하는 대신,
MLP 하나로 연속적인 장면을 파라미터화
2. 카메라 광선을 따라 샘플링하며,
각 지점의 색 + 밀도를 누적하여 픽셀 색 계산 (Volume Rendering)

9.2 3D Mesh와 Polygon 모델링

- 3D Mesh (폴리곤 메시)는 **삼각형 또는 다각형**으로 3차원 표면을 모델링하는 기법
- **Vertex Table, Edge Table, Surface Table**을 이용한 구조적 표현이 가능
- 의료 영상(두뇌 표면 등)이나 컴퓨터 그래픽에서 많이 활용

▶ 3D polygon mesh modelling



출처: <https://www.youtube.com/watch?v=ZdITviTD3VM>

<그림 설명>

- 해당 이미지는 **3D Polygon Mesh Modeling**의 기본 개념과 그것이 실제 3D 렌더링 파이프라인에서 어떻게 작동하는지 설명

[왼쪽: 3D 폴리곤 메시(Polygon Mesh)의 구조]

- 구성 요소
 - **Vertex Table (정점 테이블)**
 - 각 정점(Vertex)은 3차원 좌표 (x, y, z) 로 구성
 - 예: V1: x1, y1, z1, V2: x2, y2, z2 ...
 - **Edge Table (간선 테이블)**
 - 두 정점을 연결하는 선분

- 예: $E1: V1, V3$ 는 정점 $V1$ 과 $V3$ 를 잇는 간선
- **Surface Table (면 테이블)**
 - 하나의 다각형 면은 여러 정점과 간선으로 구성
 - 예: $S1: V1, V2, V3$ 는 세 정점으로 구성된 삼각형
 - 또는 $S1: E1, E2, E3$ 처럼 간선 기반 정의도 가능

[오른쪽: 정점 정보를 활용한 실제 렌더링 프로세스]

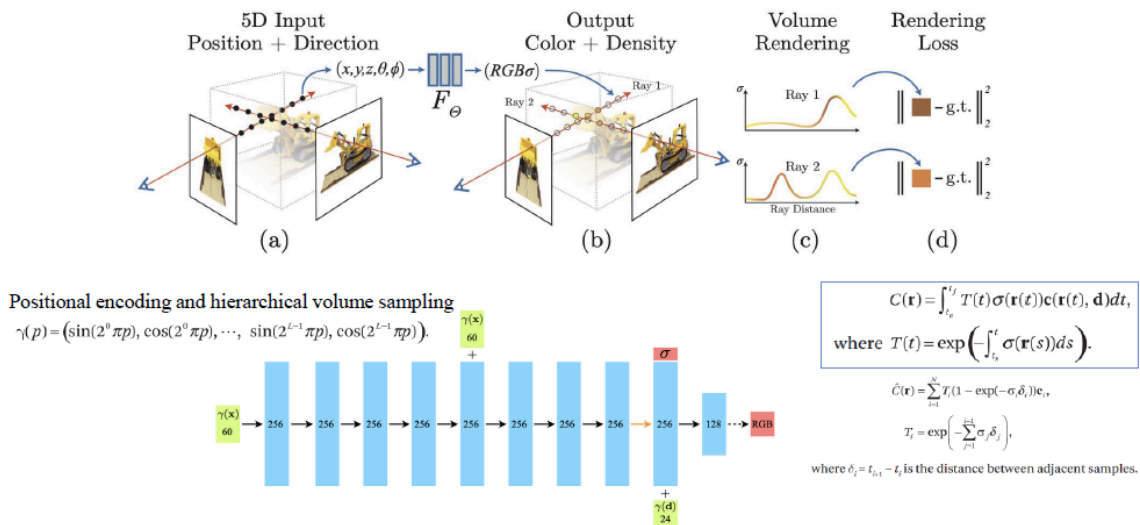
정점(Vertex) 정보를 활용하여, 어떻게 최종 이미지(렌더링 결과)가 생성되는지 알려주는 예시

- **정점 속성 (Vertex Attribute Table)**
 - **Position** (x, y, z): 3D 좌표
 - **Normal**: 표면의 방향 벡터 (조명 계산에 사용)
 - **Texture Coordinate**: 텍스처 맵핑을 위한 2D 좌표 (u, v)
 - **z-depth**: 깊이 정보 (Z-buffering에서 사용됨)
 - **color**: 정점에 부여된 색상 정보
- **변환 행렬 곱셈**
 - 정점의 위치 (x, y, z) 는 **모델** → **뷰** → **클립 공간**으로 변환되어 최종 화면에 표시
 - 하단의 수식은 **행렬 곱을 통한 위치 변환**을 설명
 - 이는 3D 위치에 **이동 (translation)** 을 적용하는 변환 예시

9.3 NeRF Architecture

- **Positional Encoding**을 통해 고주파 정보를 학습 가능하도록 처리
- Hierarchical volume sampling (코스-투-파인 방식)을 통해 연산 효율성과 정밀도 향상

▶ NeRF architecture



<그림 설명>

- NeRF의 핵심 개념인 **5D 입력** → **MLP** → **RGB/밀도 출력** → **볼륨 렌더링** → **학습의 흐름**을 설명
- NeRF는 3D 씬의 연속적 표현을 학습하고, 카메라에서 바라보는 각 방향의 픽셀 색을 예측
- 3D 위치와 시선 방향을 입력받아, RGB와 밀도(σ)를 출력하고 볼륨 렌더링을 통해 이미지를 생성

A. 5D Input: Position + Direction

1. 입력: 5차원 벡터($\mathbf{x}, \mathbf{y}, \mathbf{z}, \theta, \phi$)
2. ($\mathbf{x}, \mathbf{y}, \mathbf{z}$): 3D 공간상의 샘플링 지점
3. (θ, ϕ): 광선(ray)의 방향

→ 이 입력값은 **Positional Encoding**을 통해 고주파 정보가 보존되도록 인코딩

B. MLP를 통한 RGB & 밀도 예측

1. 인코딩된 위치 정보 → MLP 통과 → 중간에서 밀도 σ 출력
2. 이후 시선 방향 정보와 합쳐져 **RGB**를 예측
3. 네트워크 구성

→ 첫 번째 MLP 블록: 위치 → 8개의 hidden layer (256차원)

→ 두 번째 블록: **시선 방향 추가 후 RGB 출력**

C. Volume Rendering

1. 하나의 픽셀은 해당 방향으로 나아가는 광선(r)을 따라 여러 점을 샘플링하여 계산
2. 해당 점의 색상과 밀도를 기반으로 **빛의 누적 흡수 및 발산 효과**를 계산

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \cdot \sigma(\mathbf{r}(t)) \cdot c(\mathbf{r}(t), \mathbf{d}) dt$$

- 여기서,
 - $T(t)$: 누적 투과율 = 해당 위치까지 얼마나 빛이 도달할 수 있는지
 - $\sigma(\mathbf{r}(t))$: 밀도 (광선이 해당 위치에서 얼마나 막히는가)
 - $c(\mathbf{r}(t), \mathbf{d})$: 해당 위치에서 나오는 색상

D. Rendering Loss

1. 예측된 픽셀 색(Ray의 누적 결과)과 Ground Truth 이미지와의 **L2 손실** 계산
2. 이 손실을 통해 NeRF는 전체 씬 구조와 색상, 밀도 분포를 학습

• Positional Encoding & MLP 구조

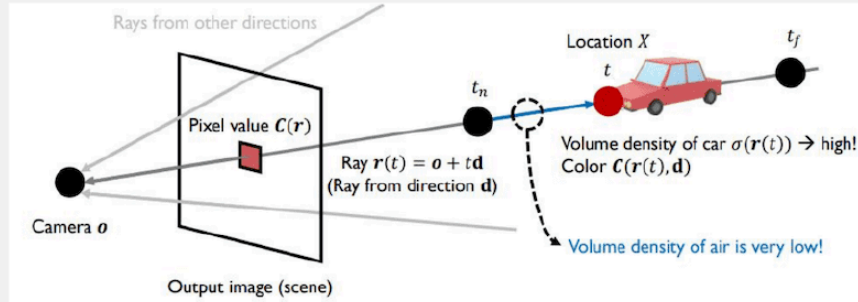
- Positional Encoding
 - 고주파 정보를 보존하기 위해 **사인/코사인 기반 함수**로 인코딩
 - 위치는 60차원으로, 방향은 24차원으로 인코딩
- MLP 네트워크
 - 8개 레이어 + Skip Connection (좌측)
 - 밀도 σ 는 중간에 출력, RGB 출력은 방향 정보를 더한 후 마지막에서 생성

9.4 Volume Rendering

▶ Volume rendering

The expected color $C(r)$. $r(t) = o + td$ with near and far bound t_n and t_f

$$C(r) = \int_{t_n}^{t_f} T(t) \sigma(r(t)) c(r(t), d) dt, \text{ where } T(t) = \exp\left(-\int_{t_n}^t \sigma(r(s)) ds\right)$$



출처: <https://cobslab.tistory.com/39>

• Volume Rendering이란?

→ **Volume Rendering**은 3D 공간을 따라 광선을 샘플링하면서, 각 지점에서의 밀도(σ)와 색상(RGB)을 이용해 최종 픽셀의 색을 계산하는 방법

◆ 수식 설명

◆ 수식

$$C(r) = \int_{t_n}^{t_f} T(t) \cdot \sigma(r(t)) \cdot c(r(t), d) dt$$

- $C(r)$: 광선 r 을 따라 보는 픽셀의 최종 색상
- $\sigma(r(t))$: 위치 $r(t)$ 에서의 **volume density** (밀도)
- $c(r(t), d)$: 위치 $r(t)$ 에서의 색상 (RGB)
- $T(t)$: 누적 투과율(**transmittance**)

$$T(t) = \exp\left(-\int_{t_n}^t \sigma(r(s)) ds\right)$$

→ 광선이 시작점 t_n 에서 지점 t 까지 얼마나 흡수되지 않고 도달했는가

<그림 설명>

• 광선과 샘플링 지점

- 카메라에서 나온 광선 $r(t) = o + td$ 가 3D 공간을 통과
- 이 광선은 여러 지점을 샘플링하며 밀도와 색을 측정

• 자동차와 공기

- 자동차가 있는 위치 t 는 밀도 σ 가 높기 때문에 색상 정보가 강하게 반영
- 반면, 공기 중은 σ 가 낮아 거의 영향을 주지 않음

• 누적 투과율

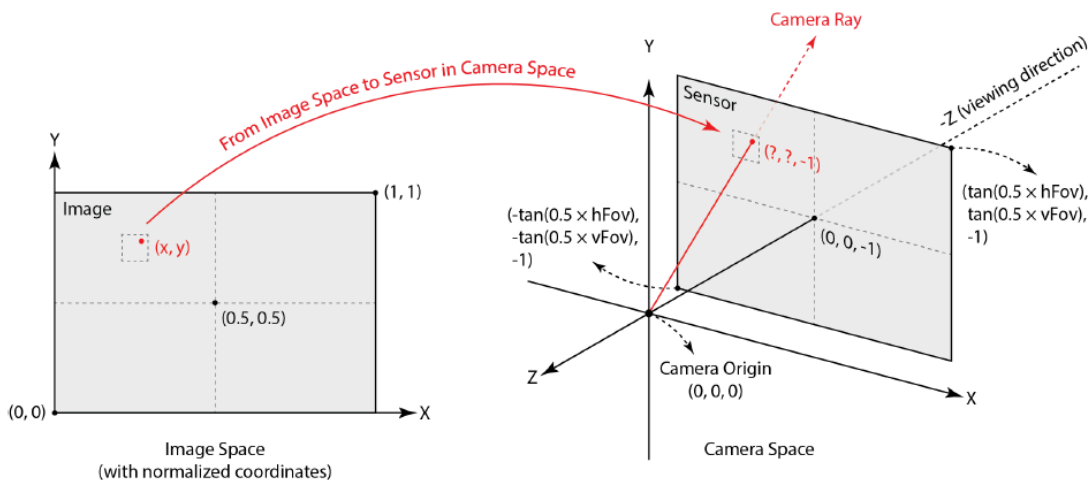
- 광선이 어떤 지점을 통과해갈수록 흡수되어 $T(t)$ 가 감소 → 깊은 곳은 더 적게 보임

• 직관적 해석

- 각 픽셀은 카메라에서 나간 **하나의 광선**에 해당
- 광선 경로를 따라 RGB 색상과 밀도를 누적하여 최종 색상 계산
- **공기처럼 투명한 영역**은 거의 영향을 안 주고
- **물체처럼 밀도가 높은 영역**은 색상에 강하게 기여

9.5 Camera Ray

► Generating camera rays



해당 이미지는 NeRF 3D 컴퓨터 그래픽스에서 사용하는 카메라 광선(Camera Ray)을 생성하는 방법을 시각적으로 설명, 2D 이미지 공간(Image Space)에서의 좌표 → 3D 카메라 공간(Camera Space) 변환하는 과정

• 목표

2D 이미지 픽셀 위치 (x, y) 에서 카메라에서 발사되는 광선(Ray)을 정의하는 것

→ 이 광선은 3D 공간의 어떤 위치를 샘플링할지를 결정합니다 (NeRF에서 매우 중요!)

[왼쪽: Image Space]

- 이미지 좌표계는 정규화된 범위인 **[0, 1]**로 구성

- 중앙은 (0.5, 0.5)
- 하나의 픽셀 위치 (x, y) 는 이 좌표계 내에서 정의

[오른쪽: Camera Space]

- 카메라의 원점은 (0, 0, 0)
- 센서(이미지 평면)는 일반적으로 $z = -1$ 평면에 위치
- 각 픽셀의 위치는 시야각(Field of View)에 따라 3D 공간으로 투영

좌표 변환 공식

이미지 좌표 (x, y) 를 카메라 센서상의 3D 점으로 매핑:

$$\text{sensor_x} = (x - 0.5) \times \tan(0.5 \times \text{hFov})$$

$$\text{sensor_y} = (y - 0.5) \times \tan(0.5 \times \text{vFov})$$

$$\text{sensor_z} = -1$$

→ 즉, 센서 평면 위의 3D 좌표는 $(?, ?, -1)$ 형태

● Red Line: Camera Ray

- 카메라 원점에서 sensor 평면의 해당 픽셀 위치까지 광선이 연결
- 이 광선의 방향 벡터가 Ray direction
- NeRF에서는 이 Ray를 따라 점을 샘플링하여 색상 및 밀도 예측에 사용

NeRF에서 왜 중요한가?

- NeRF는 광선 단위로 3D 씬을 샘플링
- 각 픽셀 → 하나의 광선 생성
- 이 광선은 (x, y, z) 위치 + viewing direction (θ, ϕ) 을 포함한 5D 입력

9.6 Multivariate Gaussian Distribution(다변량 가우시안 분포)

Multivariate Gaussian pdf

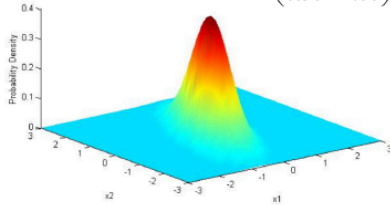
For **d dimensions**, the Gaussian distribution of a vector $x = (x^1, x^2, \dots, x^d)^T$ is defined by:

$$N(x|\mu, \Sigma) = \frac{1}{(2\pi)^{d/2} \sqrt{|\Sigma|}} \exp\left(-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)\right)$$

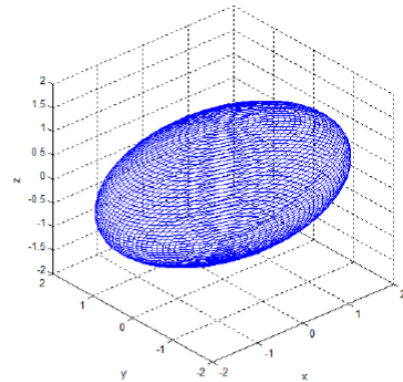
where μ is the mean and Σ is the covariance matrix of the Gaussian.

2D Gaussian

Example: $\mu = (0,0)^T$ $\Sigma = \begin{pmatrix} 0.25 & 0.30 \\ 0.30 & 1.00 \end{pmatrix}$



3D Gaussian



• 다변량 정규분포(Multivariate Gaussian)

d 차원의 벡터 $\mathbf{x}$ 에 대해 평균 μ , 공분산 행렬 Σ 를 기반으로 확률 밀도 함수(PDF)를 정의

$$N(\mathbf{x}|\mu, \Sigma) = \frac{1}{(2\pi)^{d/2} \sqrt{|\Sigma|}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)\right)$$

- μ : 평균 벡터 (분포의 중심)
- Σ : 공분산 행렬 (분포의 방향, 모양, 크기를 결정)

◆ 왼쪽 하단: 2D Gaussian 예시

- 평균 $\mu = (0, 0)^T \rightarrow$ 중심은 원점
- 공분산 행렬 $\Sigma = \begin{bmatrix} 0.25 & 0.3 \\ 0.3 & 1.0 \end{bmatrix}$
 $\rightarrow x_1$ 과 x_2 가 상관관계 있음 (0.3)
- 분포는 **종 모양(bell shape)**이고 비대칭적인 타원 형태

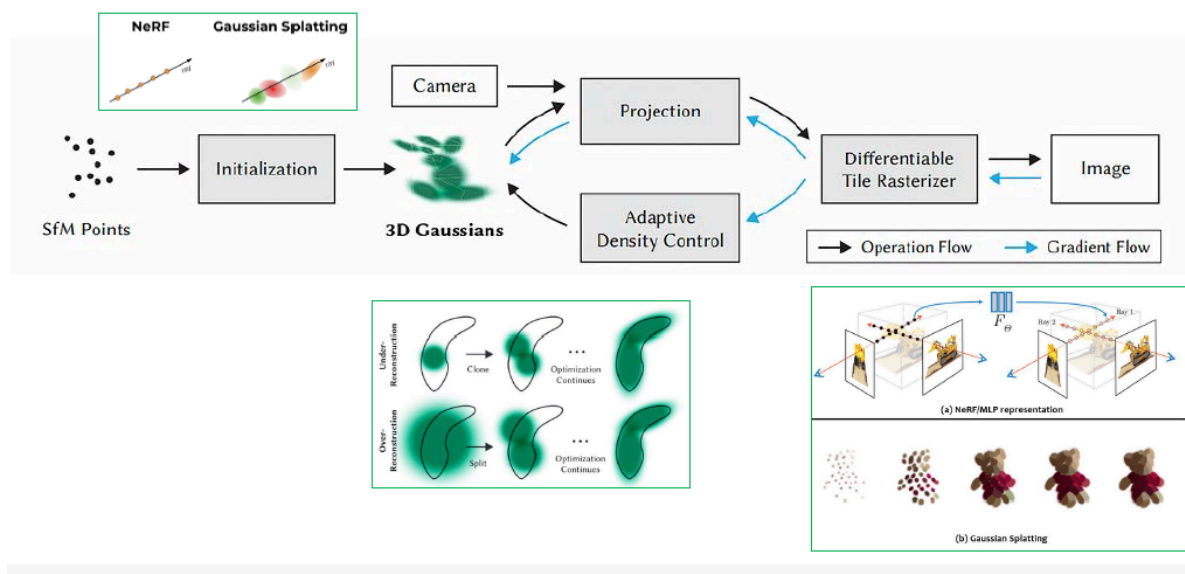
◆ 공분산의 양수 값 $\rightarrow x_1, x_2$ 가 서로 같이 증가/감소

◆ 오른쪽: 3D Gaussian (등고선 형태)

- 3차원 공간에서의 분포는 타원체(Ellipsoid) 형태
- 내부의 등고선은 동일 확률 밀도를 갖는 영역을 나타냄
- 공분산 행렬의 값에 따라:
 - 타원의 기울기 방향 (회전)
 - 길이 (축 방향 분산)
 - 납작함 or 뾰족함 (분산값 크기)
- NeRF나 Gaussian Splatting에서는 이러한 **3D Gaussian 분포**를 시각 요소로 사용
 - 물체나 광원, 밀도 등을 모델링

9.7 Gaussian Splatting

Gaussian splatting



해당 이미지는 최근 3D 신(scene) 재구성 및 렌더링 기법 중 하나인 **Gaussian Splatting**의 전체적인 파이프라인을 시각화한 것입니다. 이 방법은 기존의 NeRF와는 다르게 **점 기반 (3D Gaussian)** 표현을 사용하여, **고속 렌더링**과 **효율적인 최적화**를 가능하게 합니다

• 핵심 아이디어

1. 3D Scene을 수많은 **3D Gaussian Primitive**로 표현
2. 카메라 평면에 가우시안을 투영하여 렌더링

3. Differentiable Rasterizer로 빠르게 학습 가능
4. NeRF보다 **10~100배 빠름**, 실시간 렌더링 가능

<동작 순서>

1. SfM Points (Structure from Motion)

- 입력으로 다수의 이미지에서 추출한 3D 포인트(SfM 결과)들을 받음
- 이 포인트들은 기본적인 장면 구조 정보를 담고 있음

2. Initialization → 3D Gaussians

- 초기 3D 포인트들에 **위치, 크기, 방향, 색상, 불확실성(공분산)** 정보를 가진 **3D 가우시안(Gaussian)** 으로 초기화
- 즉, 각 포인트는 단순한 점이 아닌, 공간에 부드럽게 분포된 확률 밀도 함수로 표현

3. Projection & 카메라 관점에서의 시야 투영

- 3D Gaussian들은 현재 카메라 뷰에서 2D 이미지 평면으로 **투영(projection)**
- 이 때, 각각의 가우시안은 이미지 상의 **타일(tile)** 단위 픽셀로 분포, 렌더링 결과에 영향을 끼침

4. Adaptive Density Control

- **Under-representation**: 이미지에 부족한 디테일 → Gaussian을 **복제(Clone)** 하여 밀도를 증가
- **Over-representation**: 너무 많은 중첩/과잉 표현 → Gaussian을 **분할(Split)** 하여 최적화
- 이 과정을 통해 **효율성 + 품질**을 동시에 확보

5. Differentiable Tile Rasterizer

- 가우시안들의 투영 결과를 미분 가능한 타일 래스터라이저가 처리해 최종 이미지를 생성
- 렌더링 뿐 아니라 학습에도 사용되며, 손실 함수를 통해 가우시안들을 최적화 함

6. 최종 이미지 출력

- NeRF보다 빠르고 정확한 이미지를 실시간에 가깝게 렌더링할 수 있음
- 아래의 그림에서는 NeRF 방식(MLP 기반)과 Gaussian Splatting 방식의 비교가 포함

9.8 Tiny-NeRF

- 특징
 - 간단한 8-layer MLP로 NeRF 원리 체험
 - 해상도/샘플링 수를 줄여 연산량 축소
 - 실습 흐름:
 1. Rays 생성
 2. 샘플링
 3. MLP 예측
 4. Volume rendering
 5. Loss 계산 → 파라미터 업데이트

10. 설명 가능한 AI(XAI, Explainable AI)

10.1 XAI(Explainable AI)란?

- XAI는 AI 모델의 예측 근거를 사람이 이해할 수 있게 설명하는 기술
- 특히 딥러닝(Black-box 모델)의 내부 판단 과정을 해석하기 위해 사용
- 목적
 - 모델의 투명성 확보
 - 의료·금융·자율주행처럼 잘못된 의사결정이 큰 영향을 미치는 분야에서 필수
 - 사용자에게 모델의 판단 이유를 설명 → 신뢰성 상승

Black-box 문제

- CNN, RNN, Transformer 등은 높은 성능을 보이지만 왜 그런 예측을 했는지 이유를 파악하기 어려움
- 이를 해결하기 위한 다양한 시각화 및 기여도 분석 기법 발전

10.2 xAI의 필요성 및 활용

- 필요성
 - 의료, 금융, 자율주행 등 설명 가능성이 중요한 고신뢰 환경에서 모델의 예측 근거가 매우 중요

- AI 모델의 예측에 대한 책임성과 신뢰성 확보
- **모델 해석력과 정확도 관계**
 - White-box 모델(예: decision tree)은 해석력이 높고 정확도는 낮을 수 있음
 - Black-box 모델(예: 딥러닝)은 정확도가 높지만 해석력은 낮음

10.4 이미지 기반 XAI 기법

- **CAM (Class Activation Map)**
 - CNN 모델에서 마지막 Convolution Layer를 이용하여 **클래스별로 활성화되는 영역을 시각화(heatmap)**
 - **제약**
 - **Global Average Pooling(GAP) + Fully Connected Layer(FC)** 구조가 필요함
 - 마지막 convolutional layer에서 나온 feature map에 대해서만 히트맵을 생성할 수 있음
 - 장단점
 - 장점 : 직관적이며 해석에 용이
 - 단점 : 구조적 제한으로 GAP 기반 아키텍처에서만 가능
- **Grad-CAM(Gradient-weighted CAM)**
 - CAM의 구조적 한계를 해결한 확장 버전
 - **모든 CNN 구조에서 사용 가능 (ResNet, VGG, EfficientNet 등)**
 - 특정 클래스에 대한 **Gradient(기울기)**를 활용하여 중요 영역 계산
 - 동작 방식
 1. 입력 이미지 → CNN 통과 → Feature Map 추출
 2. 예측 클래스 score를 **마지막 conv layer의 feature map**에 대해 미분
 3. Gradient의 평균값을 **가중치(α_k)**로 사용
 4. Feature map과 가중치를 합산하여 heatmap 생성
 - 특징
 - CAM보다 훨씬 **범용적**

- 기존 모델 구조 수정 필요 없음
- **Saliency Map (Vanilla Gradient)**
 - 출력값을 입력 이미지에 대해 **미분한 Gradient의 절댓값**을 통해 입력 픽셀 중요도 시각화 기법
 - 특징
 - 입력에 민감한 위치(픽셀)를 강조하지만 노이즈가 많고 해석이 직관적이지 않을 수 있음
 - 입력값의 변화가 출력에 미치는 영향을 통해, 네트워크가 어떤 특징에 주목하는지 알 수 있음
- **Integrated Gradient(IG)**
 - ReLU 등 비선형 구간에서 Gradient가 0이 되는 문제(vanishing gradient) 방지
→ 단순 gradient 방식보다 **공정한 기여도 계산** 가능
 - 동작 방식
 1. **Baseline 이미지** 설정 (예: 전체가 0인 검은 이미지)
 2. Baseline → Input 사이를 여러 단계로 나눔($a=0 \rightarrow 1$)
 3. 각 단계에서 gradient를 계산
 4. Gradient를 **누적(적분)** → 최종 기여도 산출
- **DeepLIFT**
 - 네트워크의 입력에 대한 기여도를 계산하는 기법
 - Gradient 기반 XAI 문제 해결(기울기 0 구간에서도 기여도 계산 가능)
 - DeepLIFT는 신경망의 예측을 계산할 때 **Baseline과 비교하여 각 입력이 얼마나 기여했는지 측정**
 - 기울기 기반 방법보다 더 정확하고 직관적인 해석을 제공

10.5 Captum을 활용한 XAI실습

- 모델 구조 (ConvNet)
 - MNIST 분류 CNN
 - Conv2d → ReLU
 - Conv2d → ReLU

- MaxPool → Dropout
- FC → Dropout → FC
- 마지막 LogSoftmax 출력
- CNN 구조에서 **XAI 기법을 적용하기 적합한 형태**

- 실습 흐름

- **모델 학습**

- Optimizer: Adadelta
- Loss: Negative Log-Likelihood Loss(NLLLoss)
- Train/Test 루프 포함

- **이미지 선정**

- MNIST test set에서 샘플 1개 로드
- 시각화를 위해 grayscale → RGB 3채널 복제(`np.tile`) 처리

- **Saliency Map 계산**

```
Saliency(model).attribute(captum_input, target=label)
```

- **Integrated Gradients**

```
IntegratedGradients(model).attribute(
    captum_input,
    target=label,
    baselines=0,
    return_convergence_delta=True
)
```

- **시각화**

Captum의 `visualize_image_attr()` 로 heatmap + 원본 이미지 overlay

10.6 개념 정리

기법	특징	장점	단점
CAM	GAP + FC 기반 모델에서 만 가능	직관적	구조적 제약

기법	특징	장점	단점
Grad-CAM	Gradient 기반	모든 CNN에서 가능	마지막 conv layer만 사용
Saliency Map	입력에 대한 Gradient	간단, 빠름	노이즈, 불안정
Integrated Gradients	Baseline → Input 경로 적분	수학적으로 공정, 안정적	계산량 많음
DeepLIFT	Baseline 대비 기여도 측정	Dead ReLU 영향 없음, 빠름	구현 복잡

11. 객체 검출(Two-Stage Detection)

11.1 컴퓨터 비전 주요 과제 (TASK) & 객체 검출의 도전 과제

[컴퓨터 비전의 주요 과제]

- **Classification**
 - 이미지 전체의 **무엇인지** 판단 (예: 고양이/개)
- **Localization**
 - 이미지 속에 **어디에 있는지** 위치(Bounding Box) 파악
- **Object Detection**
 - 이미지 안에 **여러 객체**를 찾고
 - **클래스 + 위치**를 함께 예측 (핵심!)
- **Instance Segmentation**
 - 객체의 픽셀 단위까지 정확하게 구분(Mask)

➡ 객체 검출은 Classification + Localization을 동시에 해결하는 더 어려운 문제

[객체 검출의 도전 과제]

- 이미지 속 객체의 규모(크기)가 다양함
- 위치·각도·조명 변화가 큼
- 여러 객체가 겹치거나 가려짐
- 하나의 이미지에 여러 개의 객체 등장

➡ 단순 분류보다 훨씬 복잡하고 계산량이 많은 문제

11.2 IoU (Intersection over Union)

- Bounding Box의 예측 정확도를 측정하는 가장 중요한 지표
- $\text{IoU} = (\text{예측 박스} \cap \text{정답 박스}) / (\text{예측 박스} \cup \text{정답 박스})$
 - $\text{IoU} = 1 \rightarrow$ 완벽한 예측
 - $\text{IoU} \geq 0.5$ 또는 0.7 기준으로 정답 여부 판단

➡ IoU는 모든 객체 검출 모델의 기본 평가 기준

11.3 NMS (Non-Maximum Suppression)

- 과정
 1. 박스들의 confidence score 기준 정렬
 2. 가장 높은 박스를 선택
 3. IoU가 높은(중복되는) 박스 제거
 4. 반복

➡ 중복 박스를 제거하고 가장 신뢰도 높은 박스를 남기는 후처리 기법

➡ 모든 객체 검출 모델에서 필수 단계

11.4 mAP (mean Average Precision)

- 객체 검출 성능을 평가하는 가장 중요한 지표
- $\text{AP} = \text{Precision-Recall}$ 곡선 면적
- $\text{mAP} =$ 모든 클래스에 대해 AP를 평균 낸 것

➡ COCO, Pascal VOC 등 주요 대회에서도 mAP가 공식 평가 점수

11.5 초기 객체 검출 방식 — Sliding Window

[초기 방법]

- 이미지를 촘촘하게 잘라서
- 모든 영역마다 분류기(CNN 혹은 SVM)를 실행

[문제점]

- 모든 위치·크기·비율을 전부 탐색해야 해서
- 연산량 폭발적 증가 $\rightarrow$ 매우 느림

➡ 이 한계를 해결하기 위해 "Region Proposal" 방식이 등장

11.6 Region Proposal

- 대표 기법: **Selective Search**
- 이미지에서 "물체일 가능성이 있는 영역"만 1~2천개 제안
- 모든 위치를 다 탐색하지 않아도 됨 → 효율성 ↑

→ 이 방식 위에 Two-Stage Detector가 만들어짐

11.7 Two-Stage Detector란?

- 검출을 두 단계로 나누어 처리하는 방식
- **Stage 1 — Region Proposal**
 - "어디를 볼지" 찾는 단계 → RPN 또는 Selective Search 등
- **Stage 2 — Classification + Bounding Box Regression**
 - "무엇인지, 박스를 어떻게 보정할지" 최종 판단
 - → 높은 정확도가 장점
 - → 대표 모델: R-CNN 계열

11.8 R-CNN (2014) — Two-Stage Detector의 시작

- 파이프라인
 1. Selective Search로 region 2000개 추출
 2. 각 영역을 잘라 CNN에 넣어 특징 추출
 3. SVM으로 분류
 4. 회귀기로 bounding box 위치 보정
- 문제
 - 2000개 region마다 CNN을 매번 돌려야 해서 **매우 느림**

→ 이후 모델 발전의 핵심 문제 포인트 제공

11.9 Fast R-CNN (2015) — ROI Pooling 혁신

- 핵심 아이디어
 1. 전체 이미지를 CNN에 **한 번만** 통과
 2. Feature Map 위에서 ROI(관심영역)만 잘라서 처리
 3. ROI Pooling으로 고정 크기(feature size)로 변환

4. FC Layer로 분류 + 박스 회귀

- 장점
 - 속도 **대폭 향상**
 - 성능은 R-CNN보다 더 높음
 - Feature Map 공유 → 효율성 최고

11.10 ROI Pooling 원리

- 문제
 - ROI는 크기가 제각각 → FC층에 넣으려면 입력 크기가 일정해야 함
- 해결
 - ROI 영역을 **격자($k \times k$)**로 나누고, 각 칸의 max값을 뽑아 고정 크기의 벡터로 변환

➡ 위치는 다르지만 크기를 고정해주는 "Pooling" 방식

➡ Fast R-CNN 성능·속도 혁신의 중심

11.11 Faster R-CNN (2015) — 완전 End-to-End Detector

- **Region Proposal Network (RPN)**
 - Selective Search 삭제
 - CNN 위에 "Proposal 생성 전용 신경망" 추가
 - Classifier + RPN이 하나의 CNN을 공유
 - 완전한 End-to-End 학습이 가능해짐
- **성과**
 - 속도 대폭 상승
 - 정확도도 크게 향상
 - 현재까지도 Two-Stage Detector의 대표 모델